



INSTITUT INFORMATIQUE SUD AVEYRON

Reclassement professionnel

Développement web back-end

Programmation Orienté Objet – POO

Concepts de base

Sommaire

1	Introduction à la POO en PHP	2
1.1	Qu'est-ce que la POO ?	2
2	Classes, objets, instances : première approche	2
2.1	Classes, objets : exemple de création	3
2.2	Propriétés et méthodes en PHP OO	4
2.2.1	Les propriétés	4
2.2.2	Les méthodes	5
2.3	Le constructeur	7
3	Encapsulation et visibilité des propriétés et des méthodes	8
4	Héritage en PHP	8
5	Les propriétés et méthodes statique en PHP	10
6	Les méthodes et les classes abstraites en PHP	12
6.1	En pratique	12
7	Les interfaces	15
7.1	Définir et implémenter une interface	15
7.2	Interface ou classe abstraite ?	18

1 Introduction à la POO en PHP

1.1 Qu'est-ce que la POO ?

La programmation orientée objet (ou POO en abrégé) correspond à une autre manière d'imaginer, de construire et d'organiser son code.

Jusqu'à présent, nous avons codé de manière procédurale, c'est-à-dire en écrivant une suite de procédures et de fonctions dont le rôle était d'effectuer différentes opérations sur des données généralement contenues dans des variables et ceci dans leur ordre d'écriture dans le script.

La programmation orientée objet est une façon différente d'écrire et d'arranger son code autour de ce qu'on appelle des objets. Un objet est une entité qui va pouvoir contenir un ensemble de fonctions et de variables.

L'idée de la programmation orientée objet va donc être de grouper des parties de code qui servent à effectuer une tâche précise ensemble au sein d'objets afin d'obtenir une nouvelle organisation du code.

Les intérêts principaux de la programmation orientée objet vont être une structure générale du code plus claire, plus modulable et plus facile à maintenir et à déboguer.

2 Classes, objets, instances : première approche

La programmation orientée objet se base sur un concept fondamental qui est que tout élément dans un script est un objet ou va pouvoir être considéré comme un objet. Pour comprendre ce qu'est précisément un objet, il faut avant tout comprendre ce qu'est une classe.

Une classe est un ensemble cohérent de code qui contient généralement à la fois des variables et des fonctions et qui va nous servir de plan pour créer des objets. Le but d'une classe va donc être de créer des objets que nous allons ensuite pouvoir manipuler.

Imaginons qu'on possède un site sur lequel les visiteurs peuvent s'enregistrer pour avoir accès à un espace personnel par exemple. Quand un visiteur s'enregistre pour la première fois, il devient un utilisateur du site.

Ici, on va essayer de comprendre comment faire pour créer le code qui permet cela. Pour information, ce genre de fonctionnalité est quasiment exclusivement réalisé en programmation orienté objet.

Qu'essaie-t-on de réaliser ici ? On veut « créer » un nouvel utilisateur à chaque fois qu'un visiteur s'enregistre à partir des informations qu'il nous a fournies. Un utilisateur va être défini par des attributs comme son nom d'utilisateur ou son mot de passe. Ces attributs vont être des variables. Ensuite, un utilisateur va pouvoir réaliser certaines actions spécifiques comme se connecter, se déconnecter, modifier son profil, etc. Ces actions vont être des fonctions.

Nous allons donc définir les attributs et actions / fonctions de notre utilisateur. Pour cela, on va créer un formulaire d'inscription sur notre site qui va demander un nom d'utilisateur et un mot de passe d'un côté, et allons devoir côté serveur récupérer ces informations et les associer à un utilisateur en précisant qu'il s'agit du nom d'utilisateur et du mot de passe. On va également définir les actions (fonctions) propres à nos utilisateurs : connexion, déconnexion, possibilité de commenter, etc.

Sur notre site, on s'attend à avoir régulièrement de nouveaux visiteurs qui s'inscrivent et donc de nouveaux utilisateurs. Il est donc hors de question de définir toutes ces choses manuellement à chaque fois.

A la place, on va plutôt créer un bloc de code qui va initialiser nos variables nom d'utilisateur et mot de passe par exemple et qui va définir les différentes actions que va pouvoir faire un utilisateur.

Ce bloc de code est le plan de base qui va nous servir à créer un nouvel utilisateur. On va également dire que c'est une classe. Dès qu'un visiteur s'inscrit, on va pouvoir créer un nouvel objet « utilisateur » à partir de cette classe et qui va disposer des variables et fonctions définies dans la classe. Lorsqu'on crée un nouvel objet, on dit également qu'on « instancie » ou qu'on crée une instance de notre classe.

Une classe est donc un bloc de code qui va contenir différentes variables, fonctions et éventuellement constantes et qui va servir de plan de création pour différents objets. Chaque objet créé à partir d'une même classe dispose des mêmes variables, fonctions et constantes définies dans la classe mais va pouvoir les implémenter différemment.

2.1 Classes, objets : exemple de création

Reprenons notre exemple précédent et créons une première classe qu'on va appeler `Utilisateur`. Bien évidemment, nous n'allons pas créer tout un script de connexion utilisateur ici, mais simplement définir une première classe très simple.

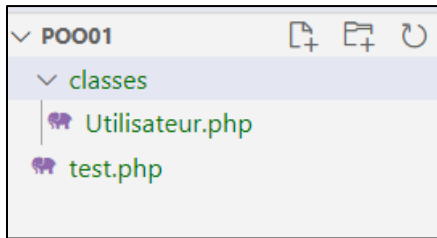
En PHP, on crée une nouvelle classe avec le mot clef `class`. On peut donner n'importe quel nom à une nouvelle classe du moment qu'on n'utilise pas un mot réservé du PHP et que le premier caractère du nom de notre classe soit une lettre ou un underscore.

Par convention, on placera généralement chaque nouvelle classe créée dans un fichier à part et on placera également tous nos fichiers de classe dans un dossier qu'on pourra appeler `classes` par exemple pour plus de simplicité.

Comme je vous l'ai dit plus haut, l'un des grands avantages de la POO se situe dans la clarté du code produit et cette clarté est notamment le résultat d'une bonne séparation du code.

On n'aura ensuite qu'à inclure les fichiers de classes nécessaires à l'exécution de notre script principal dans celui-ci grâce à une instruction `require` par exemple.

On va donc créer un nouveau fichier en plus de notre fichier principal `test.php` qu'on va appeler `Utilisateur.php`.



Dans ce fichier de classe, nous allons donc pour le moment tout simplement créer une classe **Utilisateur** avec le mot clef **class**.

```
<?php
class Utilisateur {
}
```

Pour le moment, notre classe est vide. Pour pouvoir l'utiliser, il faut l'inclure dans notre fichier principal **test.php** avec l'instruction **require**. Nous allons ensuite créer une nouvelle **instance** de la classe avec l'instruction **new** et l'affecter à une variable.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>PHP - POO - bases</title>
</head>
<body>
    <h1>Les bases de la POO en PHP</h1>
    <?php
        require 'classes/Utilisateur.php';
        $utilisateur1 = new Utilisateur();
    ?>
</body>
</html>
```

Chaque **instance** de la classe **Utilisateur** est une copie et a sa propre existence.

2.2 Propriétés et méthodes en PHP OO

2.2.1 Les propriétés

Dans le précédent chapitre, nous avons créé une classe vide que nous avons pu instancier. Nous allons maintenant ajouter des variables à notre classe. Les variables dans une classe se nomment des **propriétés** afin de les différencier des variables classiques.

```
<?php
class Utilisateur {
    public $user_name;
    public $user_pwd;
}
```

Une propriété est une variable en POO et se déclare donc de la même manière et doit respecter les mêmes contraintes.

Le mot clé **public** signifie que l'on va pouvoir accéder à ces propriétés depuis l'intérieur ou l'extérieur de la classe. Nous reviendrons sur cette notion plus tard.

Utilisons ces propriétés dans notre code.

```
<?php
    require 'classes/Utilisateur.php';

    $clement = new Utilisateur();
    $mathilde = new Utilisateur();

    $clement->user_name = "Clément";
    $clement->user_pwd = "secret";

    $mathilde->user_name = "Mathilde";
    $mathilde->user_pwd = 1234;
?>
```

Pour accéder aux propriétés, nous utilisons le symbole **->** qui est aussi appelé opérateur objet. Notez que dans ce cas le signe **\$** n'est pas utilisé.

Il est généralement considéré comme mauvaise pratique de laisser des propriétés de classes directement accessibles de l'extérieur de la classe. On préférera créer des méthodes spécifiques (**getters**, **setters**) pour manipuler ces propriétés.

2.2.2 Les méthodes

Il est possible de déclarer des fonctions à l'intérieur d'une classe. Dans ce cas les fonctions se nomment **méthodes**.

Dans notre classe Utilisateur, nous allons créer des **méthodes** pour pouvoir accéder à nos **propriétés**.

```
<?php
class Utilisateur
{
    private $user_name;
    private $user_pwd;

    public function getNom(): string {
        return $this->user_name;
    }

    public function setNom(string $nom) {
        $this->user_name = $nom;
    }

    public function setPwd($pass) {
        $this->user_pwd = $pass;
    }
}
```

Avec le `private`, les propriétés sont maintenant inaccessibles de l'extérieur de la classe. Cette pratique permet de contrôler l'accès à nos propriétés.

Les méthodes `getNom()`, `setNom()` et `setPwd()` sont des accesseurs (`getters` et `setters`). Commencer le nom de la méthode par `get` ou `set` permet de clarifier le script. Ceci est une convention.

Le mot clé `$this` sert à faire référence à l'objet (l'instance) en cours d'utilisation.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>PHP - POO - bases</title>
</head>
<body>
  <h1>Les bases de la POO en PHP</h1>
  <?php
    require 'classes/Utilisateur.php';

    $clement = new Utilisateur();
    $mathilde = new Utilisateur();

    $clement->setNom('Clément');
    $clement->setPwd("secret");

    $mathilde->setNom("Mathilde");
    $mathilde->setPwd(1234);

    echo $mathilde->getNom() . '<br>';
    echo $clement->getNom();

  ?>
</body>
</html>
```



On note que le `$this` permet de faire référence à un objet en particulier.

2.3 Le constructeur

Le constructeur d'une classe est une méthode qui va être appelée (exécutée) automatiquement à chaque instantiation de la classe. Ce constructeur se note `__construct()` (2 underscores).

A noter : Il n'est pas possible de déclarer plusieurs constructeur en PHP mais il est possible d'utiliser des arguments par défaut.

Si nous décidons qu'un utilisateur doit obligatoirement être créer avec un nom et un pwd et qu'il est interdit de modifier ces propriétés, nous pouvons réécrire notre classe de cette manière.

```
<?php
class Utilisateur
{
    private $user_name;
    private $user_pwd;

    public function __construct($n, $p) {
        $this->user_name = $n;
        $this->user_pwd = $p;
    }

    public function getNom(): string {
        return $this->user_name;
    }
}
```

Et nous l'utilisons comme suit.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>PHP - POO - bases</title>
</head>
<body>
    <h1>Les bases de la POO en PHP</h1>
    <?php
        require 'classes/Utilisateur.php';

        $clement = new Utilisateur('Clément', "secret");
        $mathilde = new Utilisateur('Mathilde', 1234);

        echo $mathilde->getNom() . '<br>';
        echo $clement->getNom();

    ?>
</body>
```



```
</html>
```

Ce code produit le même résultat que le précédent.

3 Encapsulation et visibilité des propriétés et des méthodes

L'encapsulation désigne le fait d'empêcher un accès direct aux propriétés et méthodes d'une classe.

L'encapsulation va nous permettre de garantir l'intégrité de la structure d'une classe en forçant l'utilisateur à passer par un chemin prédéfini pour modifier une donnée.

Afin de garantir ce principe, PHP met à notre disposition trois niveaux de visibilité :

- Public : l'élément est accessible partout
- Private : l'élément n'est accessible que depuis la classe
- Protected : l'élément est accessible depuis la classe et ses classe dérivées (héritage)

4 Héritage en PHP

Un des grands intérêts de la POO est qu'on va pouvoir rendre notre code très modulable, ce qui va être très utile pour gérer un gros projet ou si on souhaite le distribuer à d'autres développeurs.

Il va être possible d'étendre une classe grâce au mot clef `extends`.

En reprenant l'exemple précédent, nous allons ajouter dans le dossier classes une classe `Admin` qui va étendre `Utilisateur`.

Voici son code :

```
<?php
class Admin extends Utilisateur{

}
```

Cette classe `Admin` ne fait qu'étendre `Utilisateur`.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>PHP - POO - bases</title>
</head>
<body>
    <h1>Les bases de la POO en PHP</h1>
    <?php
        require 'classes/Utilisateur.php';
        require 'classes/Admin.php';
```

```
$clement = new Utilisateur('Clément', "secret");
$mathilde = new Utilisateur('Mathilde', 1234);
$admin = new Admin('Marcel l\'admin', 'topsecret');

echo $mathilde->getNom() . '<br>';
echo $clement->getNom() . '<br>';
echo $admin->getNom();

?>
</body>
</html>
```



La classe `Admin` hérite des méthodes de la classe `Utilisateur` quelle étend.

Nous allons pouvoir surcharger des méthodes de la classe mère dans la classe fille et ce sera la bonne méthode qui sera appelée.

```
<?php
class Utilisateur
{
    protected $user_name;
    protected $user_pwd;

    public function __construct($n, $p) {
        $this->user_name = $n;
        $this->user_pwd = $p;
    }

    protected function getNom(): string {
        return 'De la classe Utilisateur -> ' . $this->user_name;
    }
}
```

```
<?php
class Admin extends Utilisateur{

    /**
     * Redéfinition de la méthode
```

```
*/  
public function getNom(): string {  
    return 'De la classe Admin -> ' . $this->user_name;  
}  
}
```

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>PHP - POO - bases</title>  
</head>  
<body>  
    <h1>Les bases de la POO en PHP</h1>  
    <?php  
        require 'classes/Utilisateur.php';  
        require 'classes/Admin.php';  
  
        $clement = new Utilisateur('Clément', "secret");  
        $mathilde = new Utilisateur('Mathilde', 1234);  
        $admin = new Admin('Marcel l\'admin', 'topsecret');  
  
        echo $mathilde->getNom() . '<br>';  
        echo $clement->getNom() . '<br>';  
        echo $admin->getNom();  
    ?>  
</body>  
</html>
```



5 Les propriétés et méthodes statique en PHP

Une propriété ou une méthode statique est une propriété ou une méthode qui ne va pas appartenir à une instance de classe ou à un objet en particulier mais qui va plutôt appartenir à la classe dans laquelle elle a été définie.

Nous allons ajouter un compteur à notre classe `Utilisateur` afin de savoir combien de fois elle a été instanciée.

```
<?php
class Utilisateur
{
    protected $user_name;
    protected $user_pwd;

    protected static $count = 0;

    public function __construct($n, $p) {
        $this->user_name = $n;
        $this->user_pwd = $p;

        self::$count++;
    }

    public function getNom(): string {
        return 'De la classe Utilisateur -> ' . $this->user_name;
    }

    public static function getCount(){
        return 'La valeur de $count est : ' . self::$count . '<br>';
    }
}
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>PHP - POO - bases</title>
</head>
<body>
    <h1>Les bases de la POO en PHP</h1>
    <?php
        require 'classes/Utilisateur.php';
        require 'classes/Admin.php';

        $clement = new Utilisateur('Clément', "secret");
        echo Utilisateur::getCount();
        echo $clement->getNom() . '<br>';

        $mathilde = new Utilisateur('Mathilde', 1234);
        echo Utilisateur::getCount();
        echo $mathilde->getNom() . '<br>';
```

```
$admin = new Admin('Marcel l\'admin', 'topsecret');  
echo Utilisateur::getCount();  
echo $admin->getNom() . '<br>';  
?>  
</body>  
</html>
```



Quelque soit le nombre d'instances de la classe **Utilisateur**, il n'existe qu'un seul compteur qui est incrémenté chaque fois que le **constructeur** est appelé.

6 Les méthodes et les classes abstraites en PHP

Une classe abstraite est une classe qui ne va pas pouvoir être instanciée directement, c'est-à-dire qu'on ne va pas pouvoir manipuler directement.

Une méthode abstraite est une méthode dont seule la signature (c'est-à-dire le nom et les paramètres) va pouvoir être déclarée mais pour laquelle on ne va pas pouvoir déclarer d'implémentation (c'est-à-dire le code dans la fonction ou ce que fait la fonction).

Dès qu'une classe possède une méthode abstraite, il va falloir la déclarer comme classes abstraites.

6.1 En pratique

Imaginons que nous souhaitons répertorier des animaux.

Notre système va comprendre :

- Une classe abstraite Animal
- Une classe concrète Chien
- Une classe concrète Pigeon

Un animal, doit savoir dire son nom (tous les animaux le peuvent), doit pouvoir se déplacer et doit avoir un cri.

Examinons les codes suivants :

```
<?php
abstract class Animal{
    protected $nom;

    public function __construct($n){
        $this->nom = $n;
    }

    abstract public function seDeplacer();

    abstract public function crier();

    public function direSonNom(){
        echo 'Mon nom est : ' . $this->nom . '<br>';
    }
}
```

```
<?php
class Chien extends Animal{

    public function seDeplacer(){
        echo 'courir <br>';
    }

    public function crier(){
        echo 'Ouaf <br>' ;
    }

}
```

```
<?php
class Pigeon extends Animal{

    public function seDeplacer(){
        echo 'voler <br>';
    }

    public function crier(){
        echo 'Croucrou <br>' ;
    }

}
```

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>PHP - POO - bases</title>
</head>
<body>
  <h1>Les bases de la POO en PHP</h1>
  <?php
    require 'abstracts/Animal.php';
    require 'abstracts/Chien.php';
    require 'abstracts/Pigeon.php';

    $pluto = new Chien('Pluto');
    $rantamplan = new Chien('Rantamplan');
    $riri = new Pigeon('Riri');

    $pluto->direSonNom();
    $pluto->crier();
    $pluto->seDeplacer();

    $rantamplan->direSonNom();
    $rantamplan->crier();
    $rantamplan->seDeplacer();

    $riri->direSonNom();
    $riri->crier();
    $riri->seDeplacer();

    $test = new Animal();
  ?>
</body>
</html>
```



7 Les interfaces

Les interfaces vont avoir un but similaire aux classes abstraites puisque l'un des intérêts principaux liés à la définition d'une interface va être de fournir un plan général pour les développeurs qui vont implémenter l'interface et de les forcer à suivre le plan donné par l'interface.

De la même manière que pour les classes abstraites, nous n'allons pas directement pouvoir instancier une interface mais devoir l'implémenter, c'est-à-dire créer des classes dérivées à partir de celle-ci pour pouvoir utiliser ses éléments.

Les deux différences majeures entre les interfaces et les classes abstraites sont les suivantes :

- Une interface ne peut contenir que les signatures des méthodes ainsi qu'éventuellement des constantes mais pas de propriétés. Cela est dû au fait qu'aucune implémentation n'est faite dans une interface : une interface n'est véritablement qu'un plan.
- Une classe ne peut pas étendre plusieurs autres classes à cause des problèmes d'héritage. En revanche, une classe peut tout à fait implémenter plusieurs interfaces.

Précision : Autant il est impossible pour une classe d'hériter de plusieurs classes à cause du « [problème du diamant](#) », l'implémentation par une classe de plusieurs interfaces ne pose aucun problème puisque les interfaces, par définition, ne peuvent que définir la signature d'une méthode et non pas son implémentation.

7.1 Définir et implémenter une interface

Définition de l'interface `Utilisateur.php`.

```
<?php
interface Utilisateur {
    // Déclaration d'une constante
    public const ABONNEMENT = 15;

    // Déclaration des signatures des méthodes
    public function getNom();
    public function setPrixAbo();
    public function getPrixAbo();
}
?>
```

Implémentation de l'interface dans la classe `Abonne.php`

```
<?php
class Abonne implements Utilisateur{
    protected $user_name;
    protected $user_region;
    protected $prix_abo;
```



```
protected $user_pass;

public function __construct($n, $p, $r){
    $this->user_name = $n;
    $this->user_pass = $p;
    $this->user_region = $r;
}

public function getNom(){
    echo $this->user_name;
}

public function getPrixAbo(){
    echo $this->prix_abo;
}

public function setPrixAbo(){
    if($this->user_region === 'Sud'){
        return $this->prix_abo = Utilisateur::ABONNEMENT / 2;
    }else{
        return $this->prix_abo = Utilisateur::ABONNEMENT;
    }
}
}

?>
```

Implémentation de l'interface dans la classe **Admin.php**

```
<?php
class Admin implements Utilisateur{
    protected $user_name;
    protected $user_region;
    protected $prix_abo;
    protected $user_pass;
    protected static $ban;

    public function __construct($n, $p, $r){
        $this->user_name = strtoupper($n);
        $this->user_pass = $p;
        $this->user_region = $r;
    }

    public function getNom(){
        echo $this->user_name;
    }

    public function getPrixAbo(){
        echo $this->prix_abo;
    }
}
```

```
public function setBan(...$b){
    foreach($b as $banned){
        self::$ban[] .= $banned;
    }
}

public function getBan(){
    echo 'Utilisateurs bannis : ';
    foreach(self::$ban as $valeur){
        echo $valeur .', ';
    }
}

public function setPrixAbo(){
    if($this->user_region === 'Sud'){
        return $this->prix_abo = Utilisateur::ABONNEMENT / 6;
    }else{
        return $this->prix_abo = Utilisateur::ABONNEMENT / 3;
    }
}

}

?>
```

Enfin, nous allons utiliser nos classes.

```
<!DOCTYPE html>
<html lang="fr">

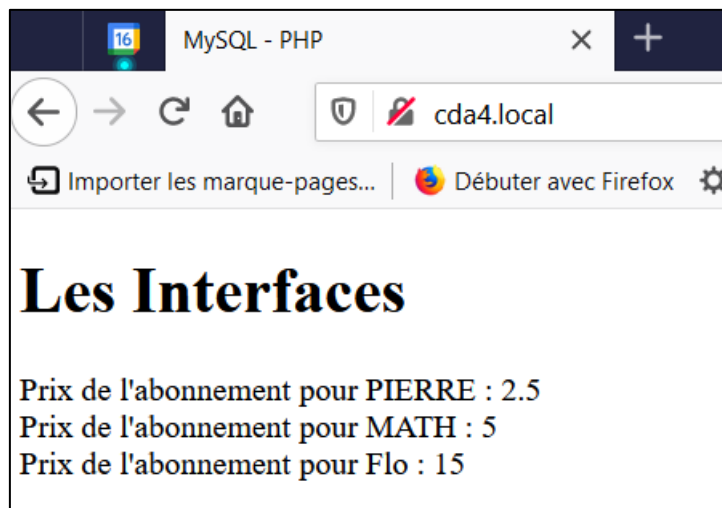
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>MySQL - PHP</title>
</head>

<body>
    <h1>Les variables</h1>
    <?php
        require 'classes/Utilisateur.php'; // l'interface
        require 'classes/Admin.php'; // classe
        require 'classes/Abonne.php'; // classe

        $pierre = new Admin('Pierre', 'abcdef', 'Sud');
        $mathilde = new Admin('Math', 123456, 'Nord');
        $florian = new Abonne('Flo', 'flotri', 'Est');

        $pierre->setPrixAbo();
        $mathilde->setPrixAbo();
        $florian->setPrixAbo();
```

```
    echo 'Prix de l\'abonnement pour ' ;  
    $pierre->getNom();  
    echo ' : ' ;  
    $pierre->getPrixAbo();  
    echo '<br>Prix de l\'abonnement pour ' ;  
    $mathilde->getNom();  
    echo ' : ' ;  
    $mathilde->getPrixAbo();  
    echo '<br>Prix de l\'abonnement pour ' ;  
    $florian->getNom();  
    echo ' : ' ;  
    $florian->getPrixAbo();  
    ?>  
</body>  
</html>
```



7.2 Interface ou classe abstraite ?

Les interfaces et les classes abstraites semblent à priori servir un objectif similaire qui est de créer des plans ou le design de futures classes.

Cependant, avec un peu de pratique, on va vite s'apercevoir qu'il est parfois plus intéressant d'utiliser une classe abstraite ou au contraire une interface en fonction de la situation.

Les interfaces ne permettent aucune implémentation mais permettent simplement de définir des signatures de méthodes et des constantes tandis qu'on va tout à fait pouvoir définir et implémenter des méthodes dans une classe abstraite.

Ainsi, lorsque plusieurs classes possèdent des similarités, on aura plutôt tendance à créer une classe mère abstraite dans laquelle on va pouvoir implémenter les méthodes communes puis d'étendre cette classe.

En revanche, si nous avons des classes qui vont pouvoir posséder les mêmes méthodes mais qui vont les implémenter de manière différente, alors il sera certainement plus adapté de

créer une interface pour définir les signatures des méthodes communes et d'implémenter cette interface pour laisser le soin à chaque classe fille d'implémenter les méthodes comme elles le souhaitent.